Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра систем штучного інтелекту



Звіт

до лабораторної роботи № 2

з дисципліни

Технології захисту інформації

Виконав:
студент КН-315
Лозовий Олег

Прийняв:
Федун В.І.

Львів – 2023

СТВОРЕННЯ ПРОГРАМНОГО ЗАСОБУ ДЛЯ ЗАБЕЗПЕЧЕННЯ ЦІЛІСНОСТІ ІНФОРМАЦІЇ

Мета роботи: ознайомитись з методами криптографічного забезпечення цілісності інформації, навчитись створювати програмні засоби для забезпечення цілісності інформації з використанням алгоритмів хешування.

Хід роботи :

Код програми:

```
from math import floor, sin
from bitarray import bitarray
import struct
from enum import Enum

def preprocessing(input):

    result = bitarray(endian="big")
    result.frombytes(input.encode("utf-8"))
    result.append(1)

    while len(result) % 512 != 448:
        result.append(0)
    result = bitarray(result, endian="big")

    length = (len(input) * 8) % pow(2, 64)
    bit_length = bitarray(endian="big")
    bit_length.frombytes(struct.pack("<Q", length))

    result.extend(bit_length)
    return result

class buffer(Enum):
    A = 0x67452301
    B = 0xEFCDAB89
    C = 0x98BADCFE
    D = 0x10325476

buffer_storge = {
        buffer.A: None,
        buffer.B: None,
        buffer.C: None,
        buffer.D: None,
    }

def rotate_left(x, n):
    return (x << n) | (x >> (32 - n))

def modul_sum(a, b):
    return (a + b) % pow(2, 32)

def buffer_update(A, B, C, D):
```

```
    modul_sum = lambda a, b: (a + b) % pow(2, 32)
    buffer_storge[buffer.A] = modul_sum(buffer_storge[buffer.A], A)
    buffer_storge[buffer.B] = modul_sum(buffer_storge[buffer.B], B)
    buffer_storge[buffer.C] = modul_sum(buffer_storge[buffer.C], C)
    buffer_storge[buffer.D] = modul_sum(buffer_storge[buffer.D], D)

def main_algo(preprocessed):

    N = len(preprocessed) // 32

    T = [
        0xd76aa478, 0xe8c7b756, 0x242070db, 0xc1bdceee,
        0xf57c0faf, 0x4787c62a, 0xa8304613, 0xfd469501,
        0x698098d8, 0x8b44f7af, 0xffff5bb1, 0x895cd7be,
        0x6b901122, 0xfd987193, 0xa679438e, 0x49b40821,
        0xf61e2562, 0xc040b340, 0x265e5a51, 0xe9b6c7aa,
        0xd62f105d, 0x02441453, 0xd8a1e681, 0xe7d3fbc8,
        0x21e1cde6, 0xc33707d6, 0xf4d50d87, 0x455a14ed,
        0xa9e3e905, 0xfcefa3f8, 0x676f02d9, 0x8d2a4c8a,
        0xfffa3942, 0x8771f681, 0x6d9d6122, 0xfde5380c,
        0xa4beea44, 0x4bdecfa9, 0xf6bb4b60, 0xbebfbc70,
        0x289b7ec6, 0xeaa127fa, 0xd4ef3085, 0x04881d05,
        0xd9d4d039, 0xe6db99e5, 0x1fa27cf8, 0xc4ac5665,
        0xf4292244, 0x432aff97, 0xab9423a7, 0xfc93a039,
        0x655b59c3, 0x8f0ccc92, 0xffeff47d, 0x85845dd1,
        0x6fa87e4f, 0xfe2ce6e0, 0xa3014314, 0x4e0811a1,
        0xf7537e82, 0xbd3af235, 0x2ad7d2bb, 0xeb86d391
    ]

    for i in range(len(T)):
        T[i] = int(T[i])

    for chunk_index in range(N // 16):

        start = chunk_index * 512
        X = [preprocessed[start + (x * 32) : start + (x * 32) + 32] for x in range(16)]
        X = [int.from_bytes(word.tobytes(), byteorder="little") for word in X]

        A = buffer_storge[buffer.A]
        B = buffer_storge[buffer.B]
        C = buffer_storge[buffer.C]
        D = buffer_storge[buffer.D]

        for i in range(64):
            if 0 <= i <= 15:
                k = i
                s = [7, 12, 17, 22]
                F = lambda x, y, z: (x & y) | (~x & z)
                temp = F(B, C, D)
            elif 16 <= i <= 31:
                k = ((5 * i) + 1) % 16
                s = [5, 9, 14, 20]
                F = lambda x, y, z: (x & z) | (y & ~z)
                temp = F(B, C, D)
            elif 32 <= i <= 47:
                k = ((3 * i) + 5) % 16
                s = [4, 11, 16, 23]
                F = lambda x, y, z: x ^ y ^ z
```

```
                temp = F(B, C, D)
            elif 48 <= i <= 63:
                k = (7 * i) % 16
                s = [6, 10, 15, 21]
                F = lambda x, y, z: y ^ (x | ~z)
                temp = F(B, C, D)

            temp = modul_sum(temp, X[k])
            temp = modul_sum(temp, T[i])
            temp = modul_sum(temp, A)
            temp = rotate_left(temp, s[i % 4])
            temp = modul_sum(temp, B)

            A = D
            D = C
            C = B
            B = temp

        buffer_update(A, B, C, D)


def convert_and_result():
    A = struct.unpack("<I", struct.pack(">I", buffer_storge[buffer.A]))[0]
    B = struct.unpack("<I", struct.pack(">I", buffer_storge[buffer.B]))[0]
    C = struct.unpack("<I", struct.pack(">I", buffer_storge[buffer.C]))[0]
    D = struct.unpack("<I", struct.pack(">I", buffer_storge[buffer.D]))[0]

    return f"{format(A, '08x')}{format(B, '08x')}{format(C, '08x')}{format(D, '08x')}".upper()


def MDFIVE(string):
  preprocessed = preprocessing(input=string)

  for buffer_type in buffer_storge.keys():
    buffer_storge[buffer_type] = buffer_type.value

  main_algo(preprocessed=preprocessed)
  algo_res = convert_and_result()

  one_out = "'" + string + "' : " + algo_res
  with open("output.txt", 'a') as file:
    file.write(one_out + '\n')
  return algo_res



if __name__ == '__main__':

  print("Menu:\n 1.Input array\n 2.Input from keyboard\n 3.Input file\n 4.Check hash")
  choise = input()

  if choise == "1":
    input = [
        "", "a", "abc", "message digest", "abcdefghijklmnopqrstuvwxyz",
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789",
        "12345678901234567890123456789012345678901234567890123456789012345678901234567890"
        ]
```

```
        with open("output.txt", 'w') as file:
            file.write('Our input array output:\n')

        for string in input:
            MDFIVE(string=string)

    elif choise == "2":
        print("\n Input text: ")
        string = input()

        with open("output.txt", 'w') as file:
            file.write('Our keyboard input output:\n')

        MDFIVE(string=string)

    elif choise == "3":
        with open("output.txt", 'w') as file:
            file.write('input.txt file output:\n')

        with open("input.txt", 'r') as file:
            file_contents = file.read()

        MDFIVE(string=file_contents[0:256])

    elif choise == "4":
        with open("hash.txt", 'r') as file:
            file_contents = file.read()

        while True:
            print("\n Input potential text: ")
            string = input()
            with open("output.txt", 'w') as file:
                file.write('Potential hash output:\n')
            maybe = MDFIVE(string=string)

            if file_contents == maybe:
                with open("output.txt", 'a') as file:
                    file.write("\nGotcha")
                    break
            else:
                with open("output.txt", 'a') as file:
                    file.write("\nThats not our input")

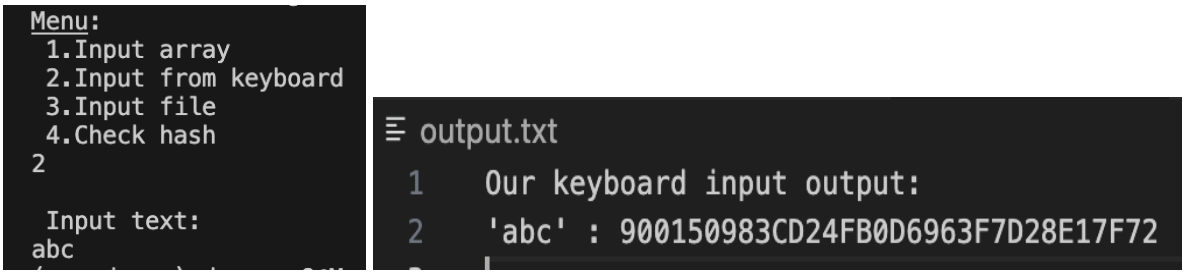
    else:
        print("\nClown?\n\nMenu:\n 1.Input array\n 2.Input from keyboard\n 3.Input file")
        choise = input()
```

Результати виконання програми:

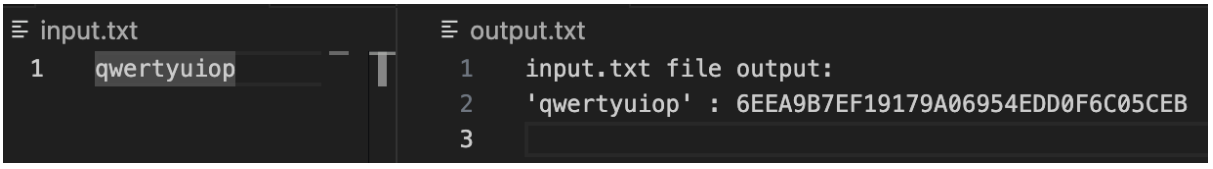
А) Масив даних

```
Our input array output:
'' : D41D8CD98F00B204E9800998ECF8427E
'a' : 0CC175B9C0F1B6A831C399E269772661
'abc' : 900150983CD24FB0D6963F7D28E17F72
'message digest' : F96B697D7CB7938D525A2F31AAF161D0
'abcdefghijklmnopqrstuvwxyz' : C3FCD3D76192E4007DFB496CCA67E13B
'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789' : D174AB98D277D9F5A5611C2C9F419D9F
'12345678901234567890123456789012345678901234567890123456789012345678901234567890' : 57EDF4A22BE3C955AC49DA2E2107B67A
```

B) Ввід з клавіатури

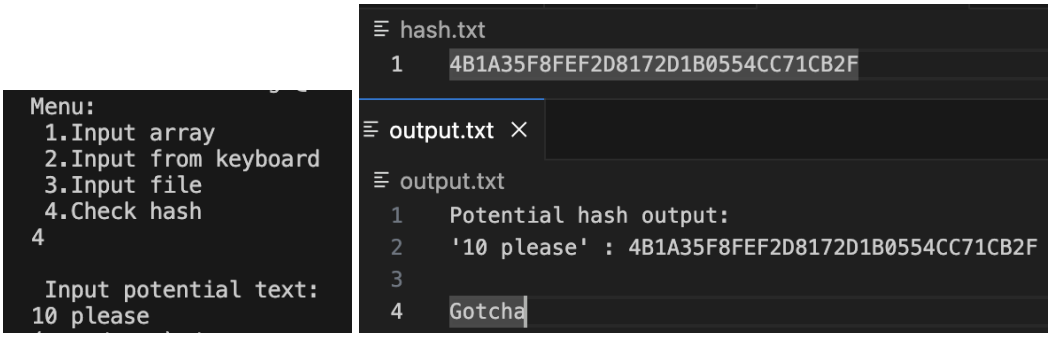


C) Ввід через файл

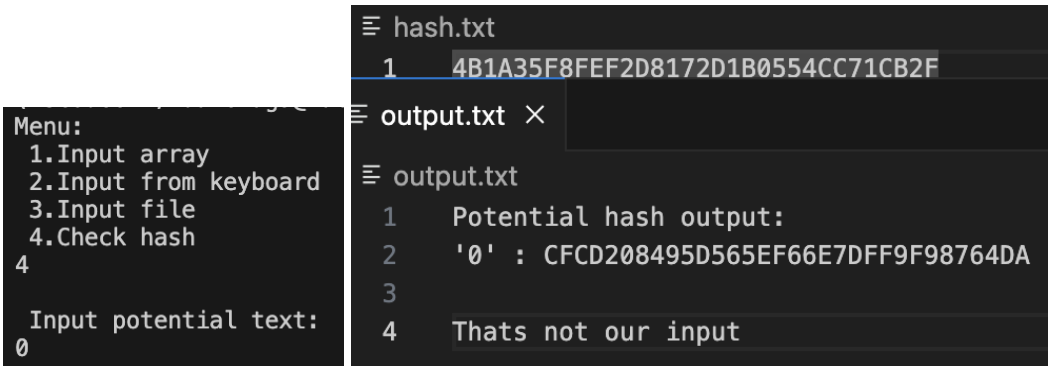


D) Перевірка цілісності

- правильний ввід



- неправильний ввід



Висновок: виконуючи дану лабораторну роботу, я ознайомився з методами криптографічного забезпечення цілісності інформації, алгоритмом MD5 та створив програму для хешування тексту методом хешування MD5.